

ЗАДАНИЕ 7

Основы МРІ, средство профилирования МРІ-Р

Ганеева Сандра гр.538

Ноябрь 2024

1 ПОСТАНОВКА ЗАДАЧИ

Задание:

Произвести итерации метода Якоби для уравнения Лапласа ($g = 0$) в 2D (двумерном) случае

Общий ход программы:

1. Подключение библиотек, параметры задачи.
2. Инициализация MPI.

Запускаем множество процессов и делим сетку на горизонтальные полосы.

`MPI_Comm_rank` — определяет ранг процесса.

`MPI_Comm_size` — определяет общее количество процессов (`size`) в группе.

3. Деление сетки и выделение памяти.

Каждый процесс обрабатывает часть сетки (горизонтальную полосу) плюс "призрачные" строки — это границы, используемые для обмена данными.

4. Обмен данными между процессами.

`MPI_Sendrecv` отправляет данные и одновременно получает. Используем "верхнюю" и "нижнюю" строки для обмена между соседними процессами.

5. Метод Якоби.

Метод Якоби - каждый элемент сетки обновляется как среднее арифметическое значений соседей. Выполняются итерации, обновляя значения в сетке.

`local_diff` суммирует разности между новой и старой сеткой.

6. Проверка сходимости.

Суммируем разности между соседними итерациями по всем процессам. `MPI_Allreduce` суммирует `local_diff` по всем процессам. Проверяем условие сходимости, если ошибка меньше `TOL`.

7. Завершение программы.

Освобождаем память и завершаем MPI.

2 Запуск

На локальной машине (Ubuntu):

Компиляция

```
mpicc sem7.c -o sem7
```

Запуск

```
mpirun -np 4 ./prog 256 1000 0.000001
```

256 - размер сетки
1000 - макс кол-во итераций Якоби
0.01 - точность
Tolerance задает критерий сходимости метода Якоби.

На Polus:

Компиляция
mpixlc sem7.c -o sem7
Запуск
mpisubmit.pl -p 4 -w 00:05 -stdout result.out ./a.out

3 ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ

AppTime: Общее время работы приложения для каждого процесса.

MPITime: Время, проведенное в MPI-коммуникациях.

MPI: Процент времени, который каждый процесс тратит на MPI-вызовы.

Результаты на 2 процессах:

App Time - 1.1 сек.

MPI Time - 0.00838 сек.

MPI: 0.76 .

Низкий процент, программа эффективно выполняет вычисления и не тратит много времени на коммуникацию между процессами

ОСНОВНЫЕ ВЫЗОВЫ MPI:

Allreduce:

Общее время: 4.6 мс

(0.42 времени приложения, 54.82 от времени, затраченного на MPI).

Sendrecv:

Процесс 1: 1.85 мс (0.17 времени приложения, 22.01 времени MPI)

Процесс 3: 1.94 мс (0.18 времени приложения, 23.17 времени MPI)

Call	Site	Time	App%	MPI%	Count	COV
Allreduce	2	4.6	0.42	54.82	2000	0.21
Sendrecv	3	1.94	0.18	23.17	1000	0.00
Sendrecv	1	1.85	0.17	22.01	1000	0.00

Результаты на 8 процессах:

App Time: 1.86 сек

MPI Time: 0.55 сек

MPI: 29.56

ОСНОВНЫЕ ВЫЗОВЫ MPI:

Allreduce:

Общее время: 25.32 мс (85.66 времени MPI).

Количество вызовов: 8000.

Sendrecv:

Процесс 3: 68.7 мс (12.49 времени MPI).

Процесс 1: 10.2 мс (1.85 времени MPI).

Call	Site	Time	App%	MPI%	Count	COV
Allreduce	2	471	25.32	85.66	8000	0.43
Sendrecv	3	68.7	3.69	12.49	7000	0.49
Sendrecv	1	10.2	0.55	1.85	7000	0.73

Результаты на 16 процессах: App Time: 3.21 сек

MPI Time: 1.25 сек

MPI: 39.11

ОСНОВНЫЕ ВЫЗОВЫ MPI:

Allreduce:

Общее время: 26.64 мс (68.11 времени MPI).

Количество вызовов: 16000.

Sendrecv:

Процесс 3: 375 мс (29.90 времени MPI).

Процесс 1: 25 мс (1.99 времени MPI).

Call	Site	Time	App%	MPI%	Count	COV
Allreduce	2	854	26.64	68.11	16000	0.75
Sendrecv	3	375	11.69	29.90	15000	0.47
Sendrecv	1	25	0.78	1.99	15000	0.73

Результаты на 32 процессах:

App Time: 6.36 сек

MPI Time: 4.2 сек

MPI 66.00

ОСНОВНЫЕ ВЫЗОВЫ MPI:

Allreduce: Время: 2.53 сек (60.22 времени MPI).

Количество вызовов: 32,000.

Операция Allreduce продолжает быть самой затратной, занимая более 60 процентов времени

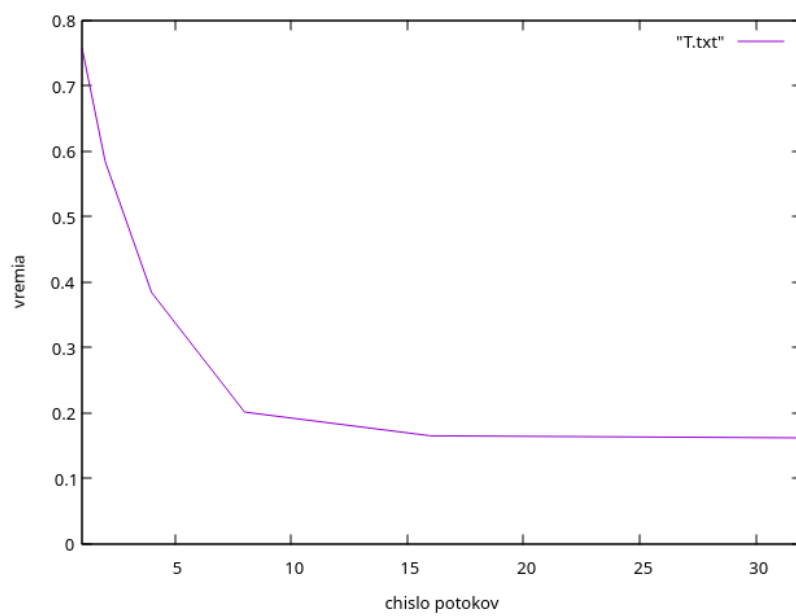
Sendrecv: Процесс 3: 1.61 сек (38.48 времени MPI).

Процесс 1: 54.7 мс (1.30 времени MPI).

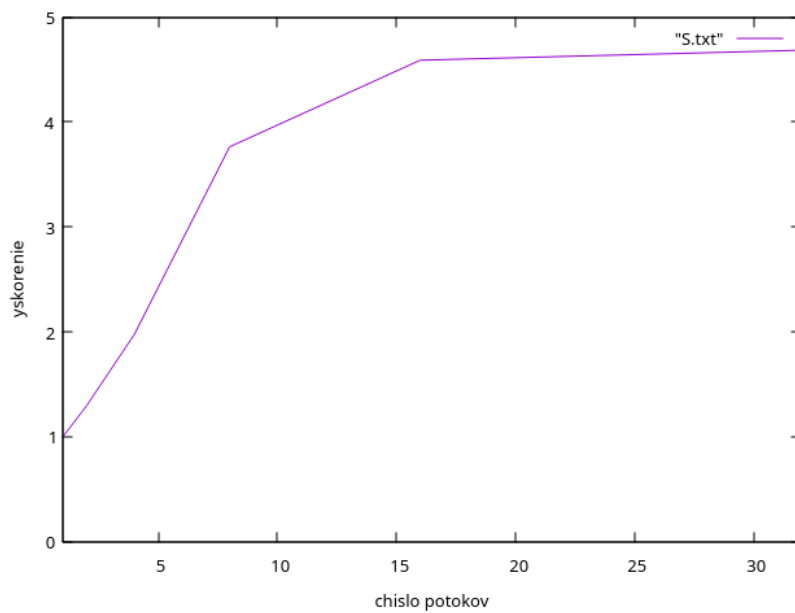
Call	Site	Time	App%	MPI%	Count	COV
Allreduce	2	2.53e+03	39.75	60.22	32000	0.46
Sendrecv	3	1.61e+03	25.39	38.48	31000	0.52
Sendrecv	1	54.7	0.86	1.30	31000	0.57

P	AppTime (c)	MPITime (c)	MPI
1	1.09	0.000957	0.09
2	1.1	0.00838	0.76
4	1.37	0.159	11.59
8	1.86	0.55	29.56
16	3.21	1.25	39.11
32	6.36	4.2	66.00

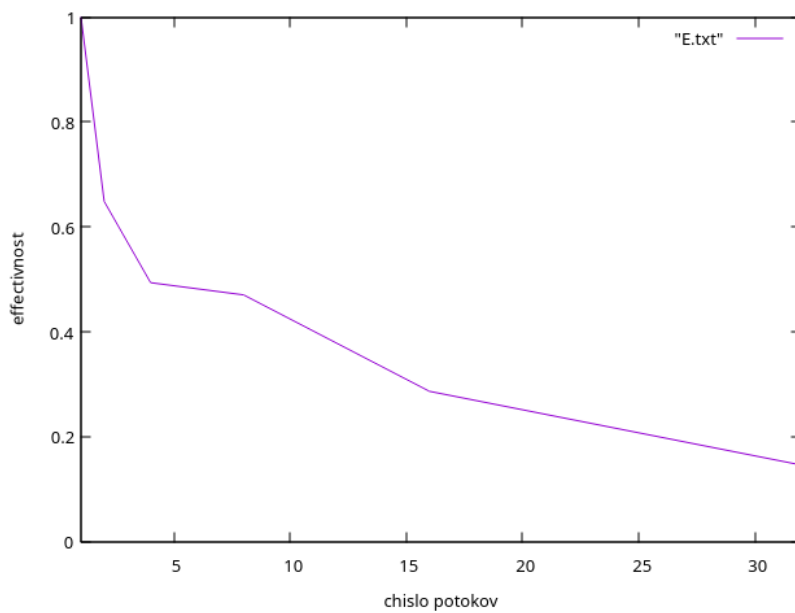
4 ГРАФИКИ



Время от числа потоков $T(P)$



Ускорение числа от потоков S (P)



Эффективность E (P)

5 ВЫВОД

Анализ времени работы $T(p)$:

Время выполнения программы уменьшается с увеличением числа процессов, что соответствует нашим ожиданиям. Начиная с 16 процессов, прирост производительности становится незначительным

Анализ ускорения $S(p)$:

Ускорение показывает, насколько быстрее работает программа по сравнению с последовательной

$S(2)=1.2973$: использование 2 процессов даёт небольшое увеличение скорости.

А вот $S(8)=3.7634$: использование 8 процессов даёт почти 4-кратное ускорение.

Анализ эффективности $E(p)$:

Эффективность снижается по мере увеличения числа процессов. Скорее всего из-за роста накладных расходов на синхронизацию и обмен данными.